



Departamento de Informática e Matemática Aplicada – DIMAp

Padrões de Resiliência com Resilience4j Parte 1

Prof. Nélio Cacho

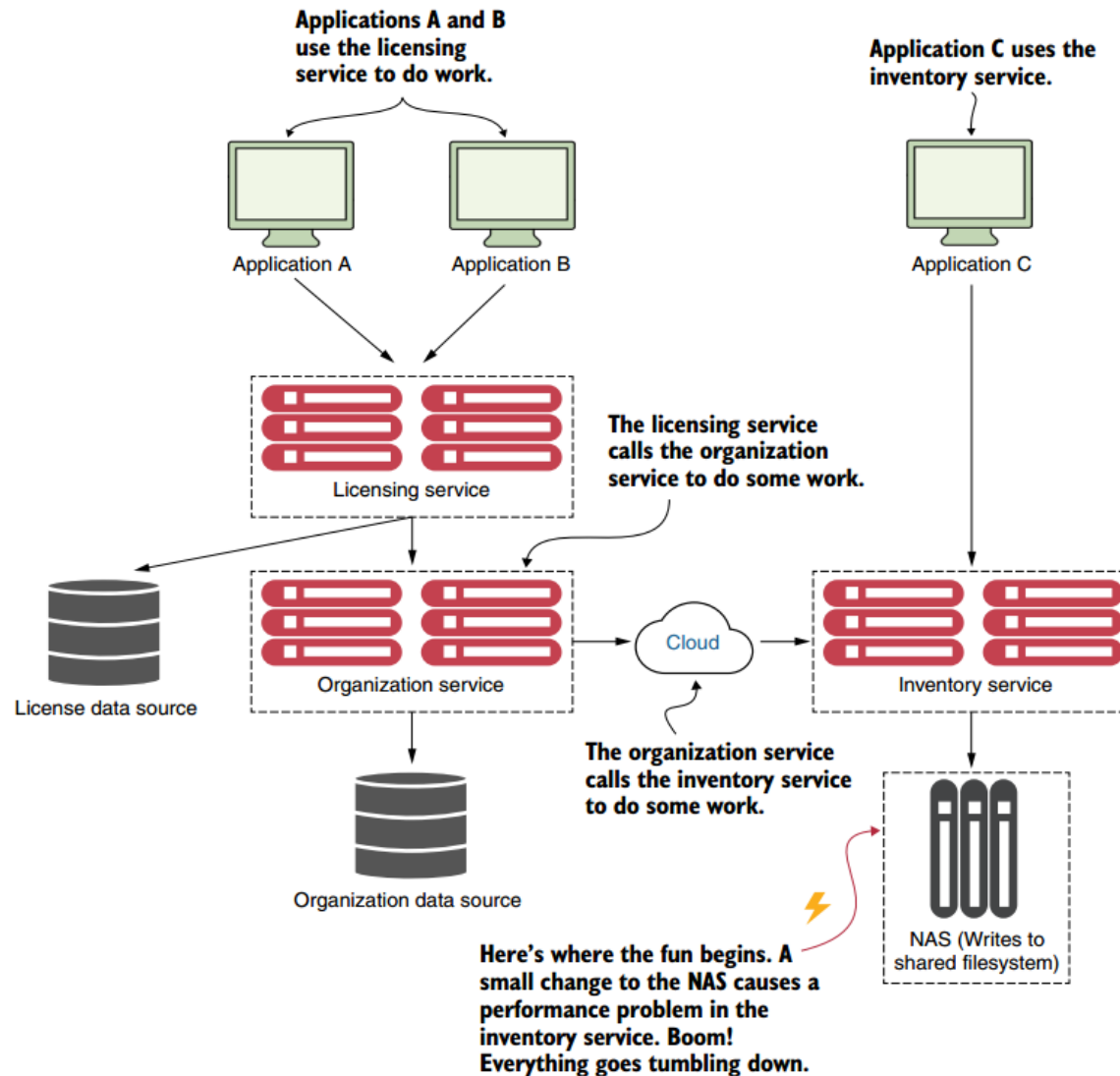
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...

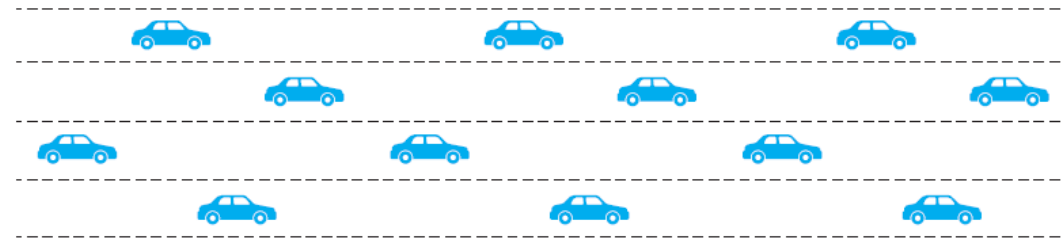


- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Por que precisamos de soluções resilientes?

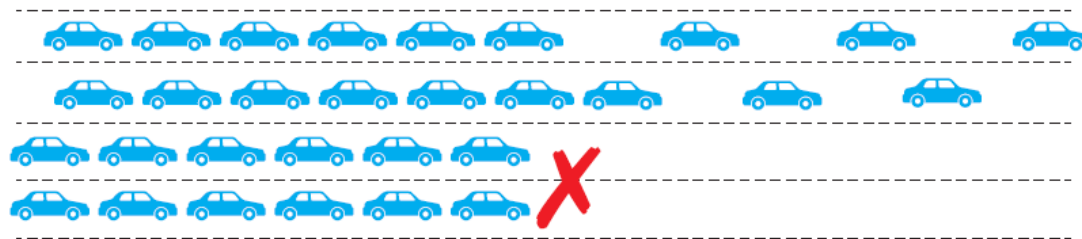


Por que precisamos de soluções resilientes?



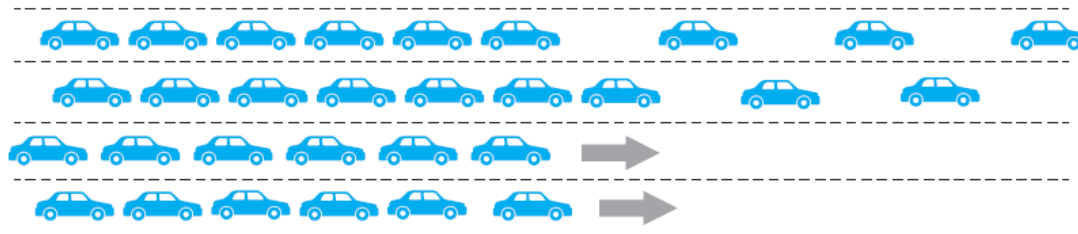
Normal traffic flow

A freeway is set up for a certain capacity, allowing traffic to flow freely as long as it is within those capacity constraints and the capacity is not disrupted.



Restricted "bandwidth"

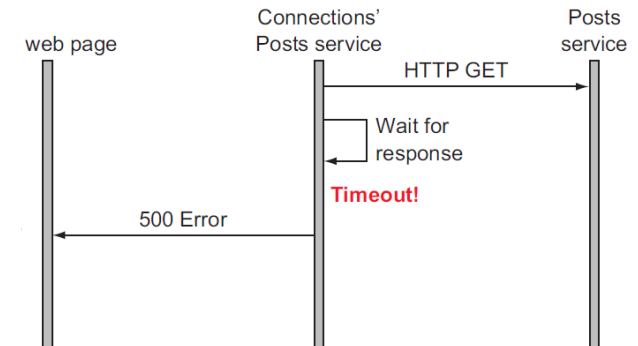
Less space between the cars means the flow in the remaining two lanes will be much slower than before. Further, with the incoming volume as before, everything comes to a standstill.



After accident cleanup

It will take some time before the stopped cars all have an opportunity to move again.

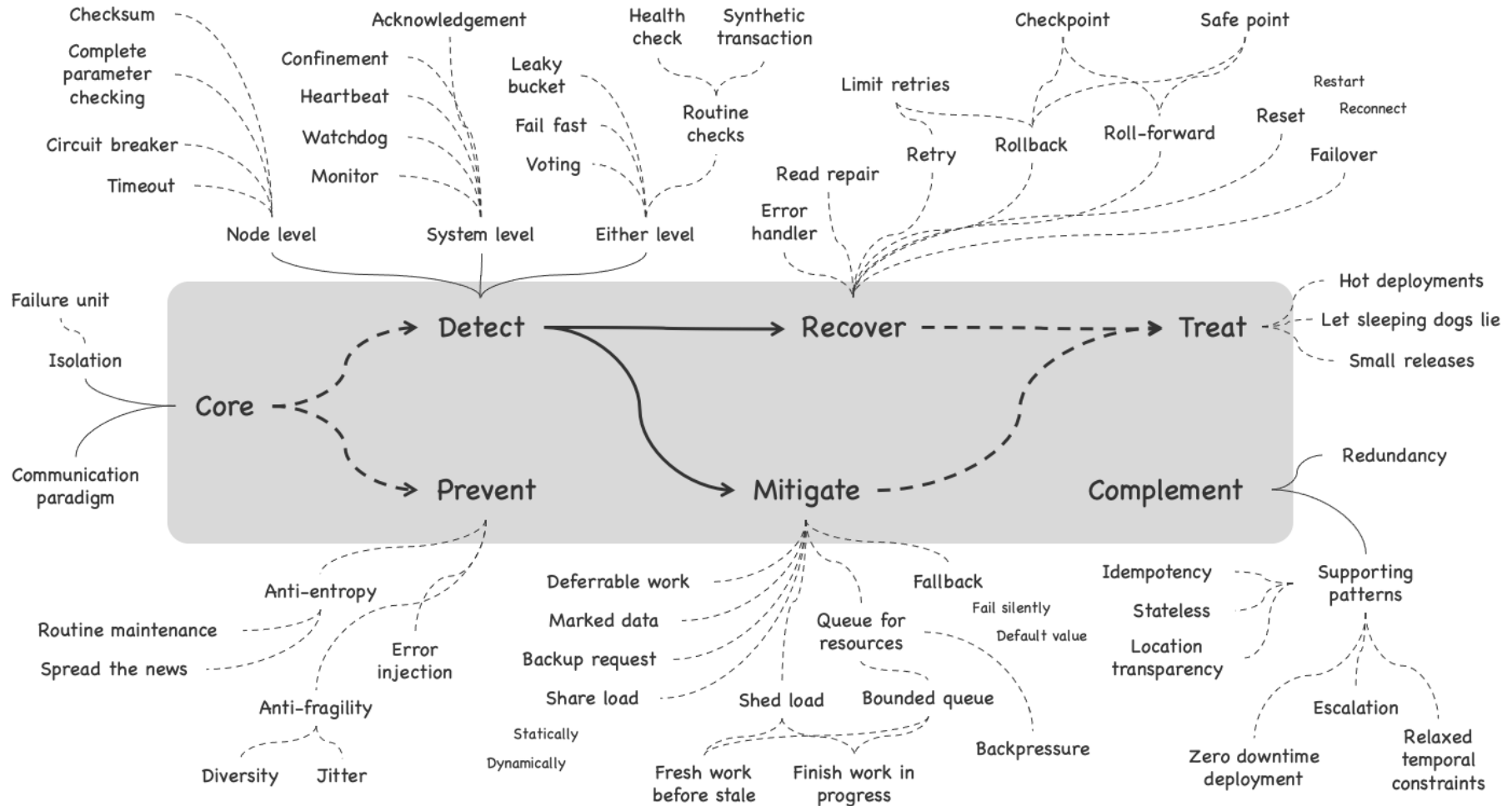
These cars are not moving for a while yet.



Por que resiliência é importante?

- **A degradação do serviço pode começar como algo intermitente e depois ganhar impulso.** A degradação do serviço também pode ocorrer apenas em pequenos surtos. Os primeiros sinais de falha podem ser um pequeno grupo de usuários reclamando de um problema até que, de repente, o contêiner da aplicação exauste seu pool de threads e entra em colapso completamente.
- **As chamadas para serviços remotos geralmente são síncronas e não interrompem uma chamada em execução por muito tempo.** Normalmente, o desenvolvedor da aplicação faz uma chamada a um serviço para realizar uma ação e aguarda o retorno do serviço. O chamador não tem conceito de um limite de tempo para evitar que a chamada ao serviço fique pendente.
- **As aplicações frequentemente são projetadas para lidar com falhas completas de recursos remotos, não com degradações parciais.** Muitas vezes, desde que o serviço não tenha falhado completamente, uma aplicação continuará chamando um serviço que está se comportando mal e não falhará rapidamente. Nesse caso, a aplicação ou serviço chamador pode se degradar graciosamente ou, mais provavelmente, travar devido à exaustão de recursos. A exaustão de recursos ocorre quando um recurso limitado, como um pool de threads ou uma conexão de banco de dados, atinge seu limite máximo, e o cliente chamador precisa aguardar até que esse recurso fique disponível novamente.

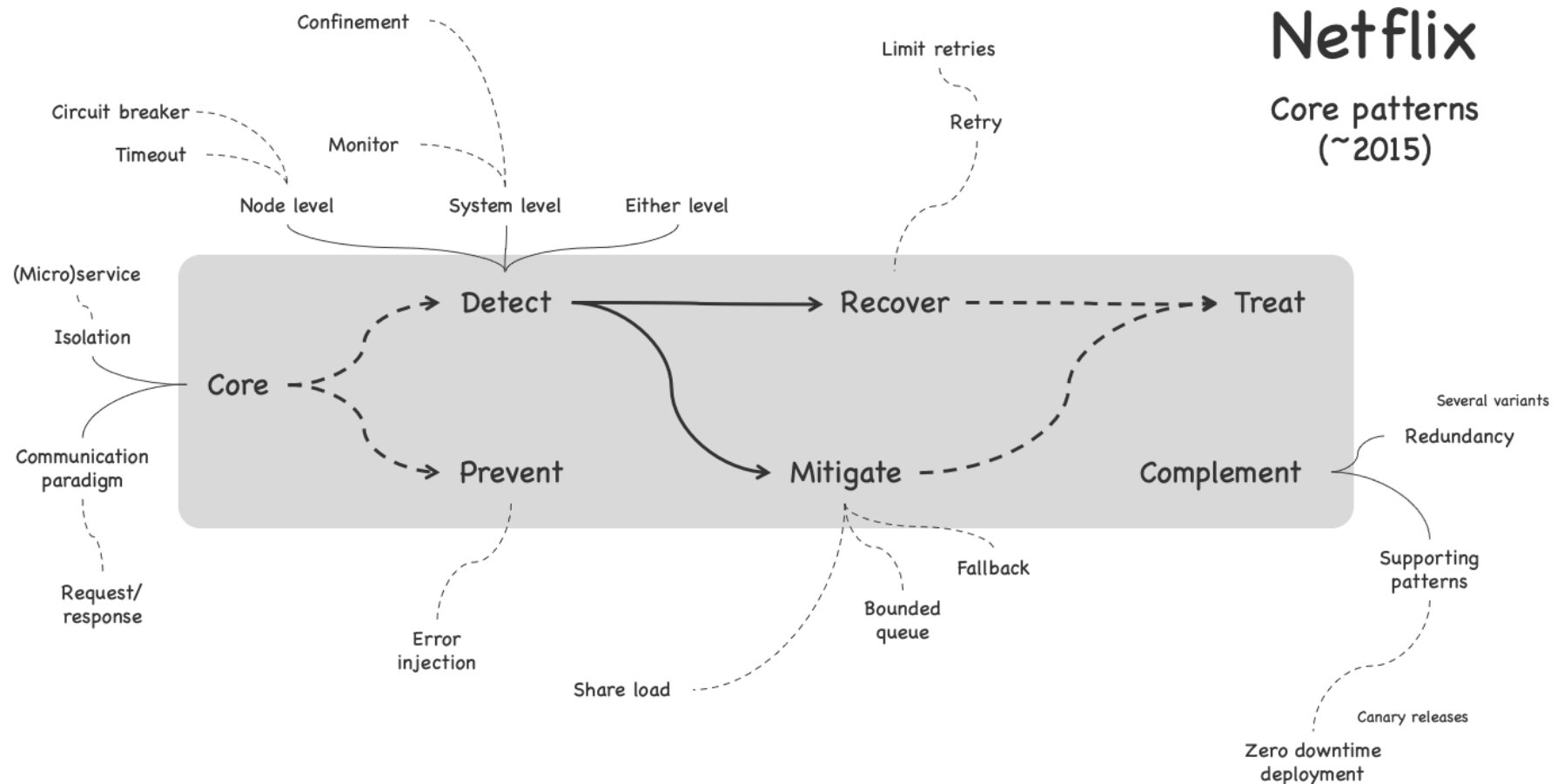
Padrões de Design Resilientes



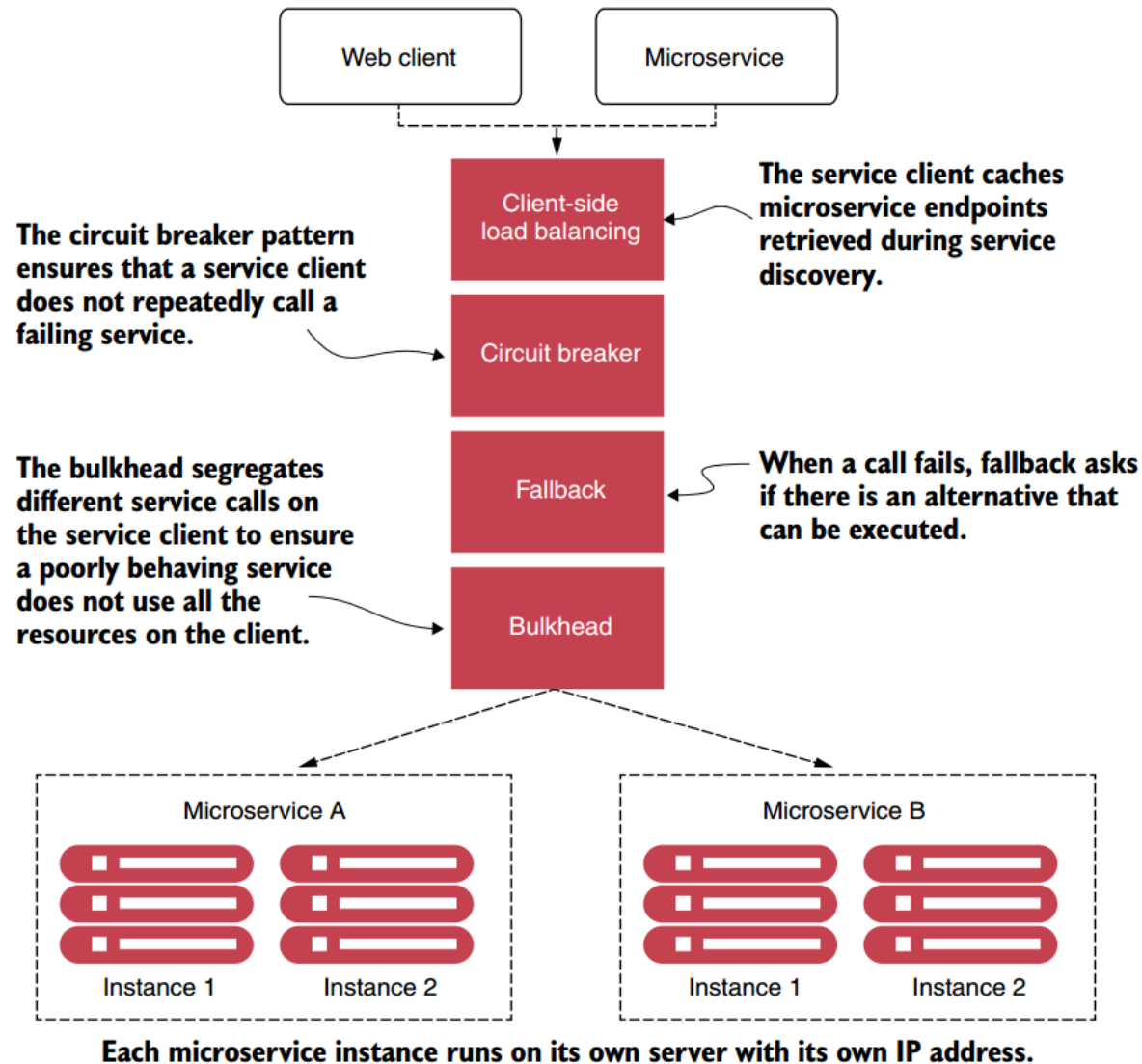
Padrões de Design Resilientes

- O caso comercial para o design de software resiliente é um pouco complicado porque a **resiliência não se trata de ganhar dinheiro, mas de não perder dinheiro**.
- Você também não perde dinheiro imediatamente se não fizer nada, mas somente se algo imprevisto acontecer, o que introduz uma probabilidade.
- Isso muitas vezes torna difícil negociar um orçamento para a resiliência, especialmente quando confrontado com pessoas que tendem a pensar apenas em dependências diretas e de curto prazo.
- **Infelizmente**, assim como a segurança, **os efeitos muitas vezes catastróficos de não se preocupar com a resiliência só se tornam óbvios quando é tarde demais**.
- Portanto, gastar pouco pode representar um risco para a vida das empresas/pessoas.
- Por outro lado, gastar demais também não faz sentido do ponto de vista econômico. Existem inúmeras opções para melhorar a resiliência de sua aplicação, mas implementar todas elas também não é útil.

Portanto, na prática, um menor conjunto de padrões são utilizados



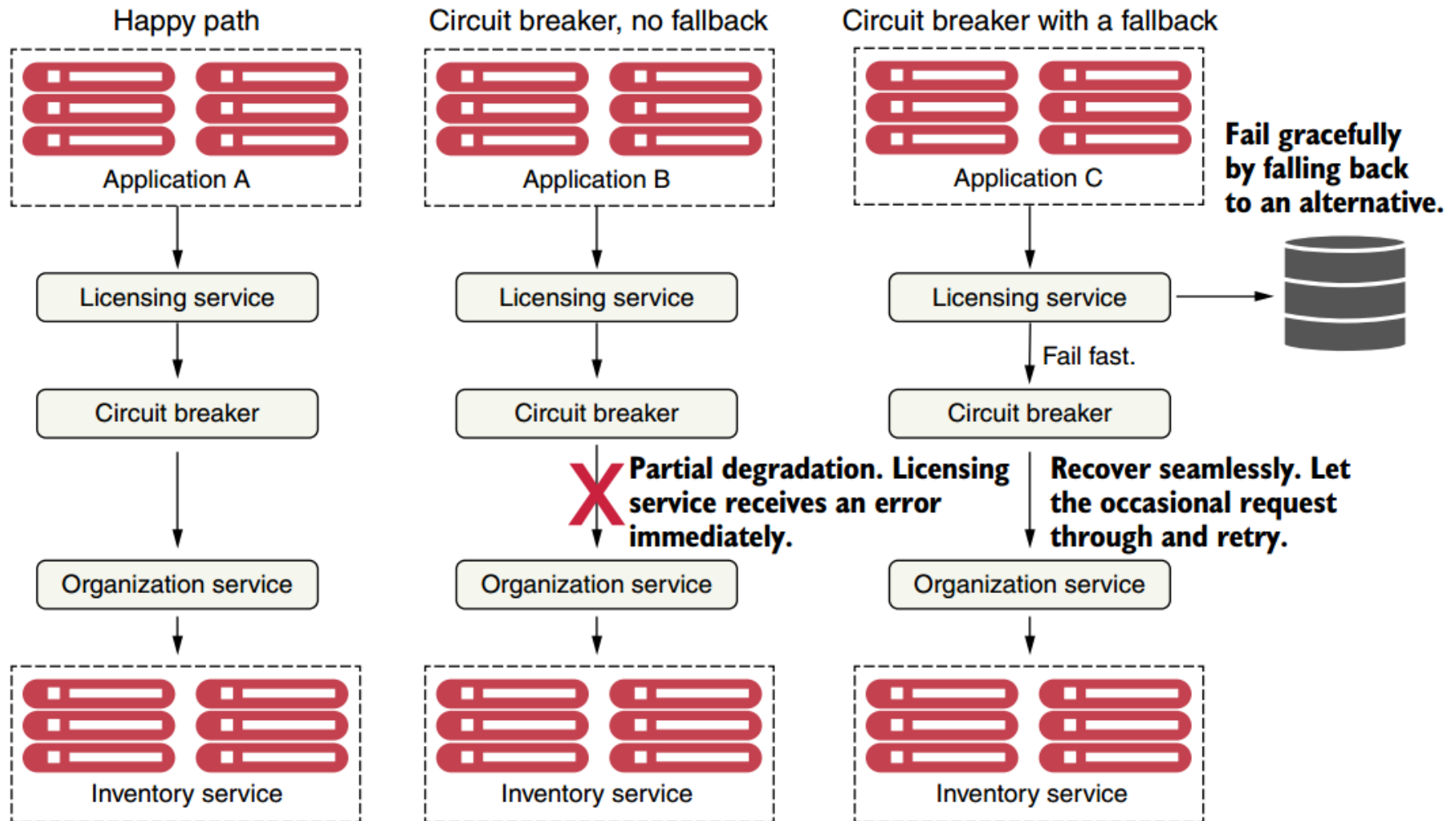
Vamos ver agora como implementar alguns padrões!!!



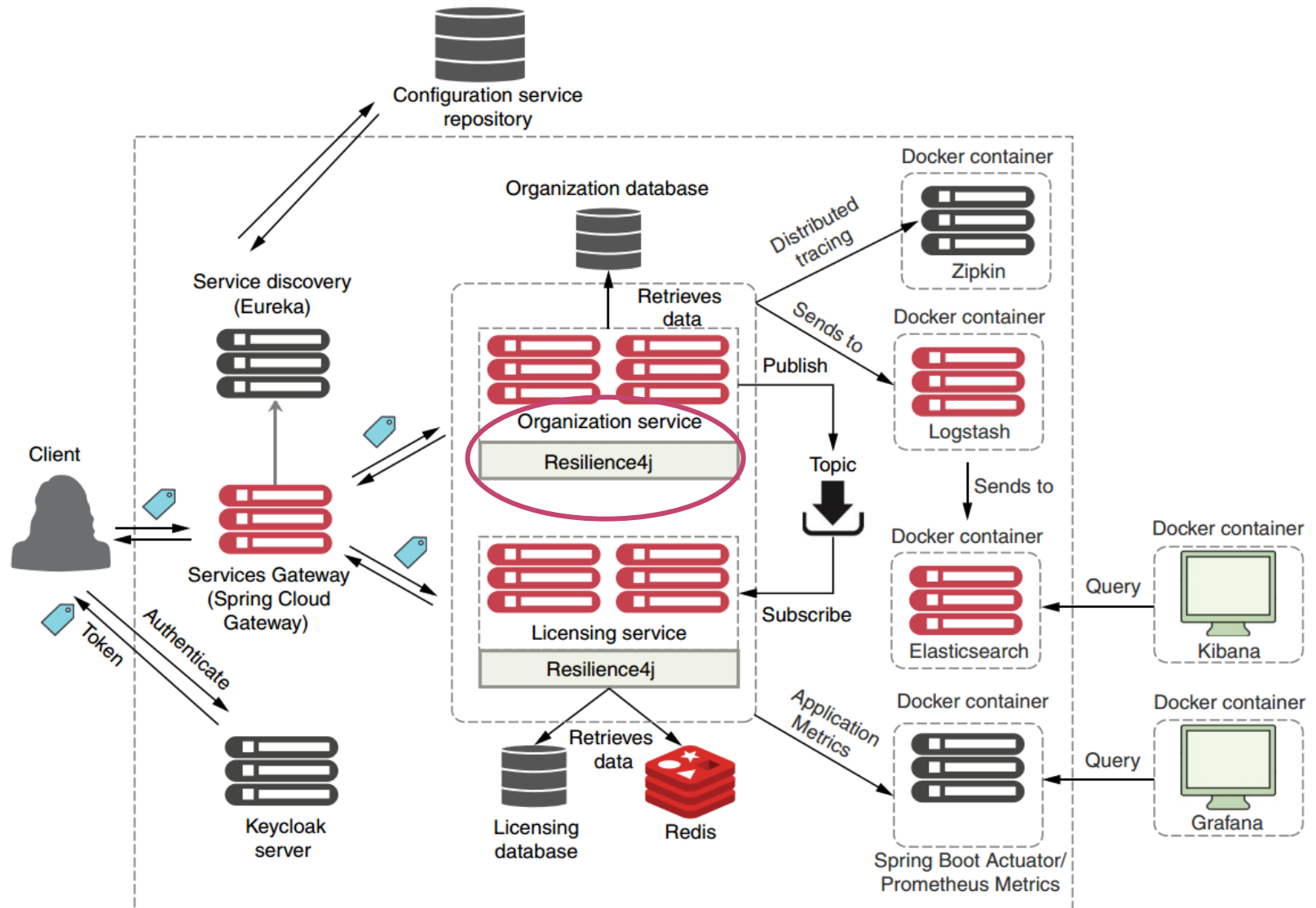
Por que padrões de resiliência no lado do cliente são importantes?

- **O balanceamento de carga no lado do cliente** envolve o cliente procurar todas as instâncias individuais de um serviço por meio de um agente de descoberta de serviço (como o Netflix Eureka) e armazenar em cache a localização física dessas instâncias de serviço.
- **Os circuit breakers monitoram chamadas remotas e evitam esperas longas em serviços.** Nesses cenários, o circuit breaker é responsável por encerrar essas conexões e monitorar se há mais chamadas falhando ou com mau comportamento.
- **O padrão de fallback**, quando uma chamada a um serviço remoto falha, em vez de gerar uma exceção, o consumidor do serviço executa um caminho de código alternativo e tenta realizar a ação por outros meios.
- **O padrão de bulkhead** envolve separar as chamadas para recursos remotos em seus próprios pools de threads e reduzir o risco de que um problema com uma chamada lenta a um recurso remoto derrube toda a aplicação.

Por que padrões de resiliência no lado do cliente são importantes?



Microservice patterns: The Big Picture



Implementando padrões via Resilience4J

- Resilience4j é uma biblioteca de tolerância a falhas inspirada no Hystrix. Ela oferece os seguintes padrões para aumentar a tolerância a falhas devido a problemas de rede ou falha de qualquer um dos nossos vários serviços:
 - **Circuit breaker (Disjuntor)** - Interrompe a realização de solicitações quando um serviço invocado está falhando.
 - **Retry (Tentativa)** - Tenta novamente um serviço quando ele falha temporariamente.
 - **Rate Limit** – Limita o número de chamadas que um serviço pode receber ou realizar para evitar overload.
 - **Bulkhead (Casco de Navio)** - Limita o número de solicitações de serviço simultâneas de saída para evitar sobrecarga.
 - **Fallback (Alternativa)** - Define caminhos alternativos para solicitações com falha.

Utilizando Resilience4J

```
<properties>
  <java.version>17</java.version>
  ....
  <resilience4j.version>2.3.0</resilience4j.version>
</properties>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>${resilience4j.version}</version>
</dependency>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-circuitbreaker</artifactId>
  <version>${resilience4j.version}</version>
</dependency>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-timelimiter</artifactId>
  <version>${resilience4j.version}</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

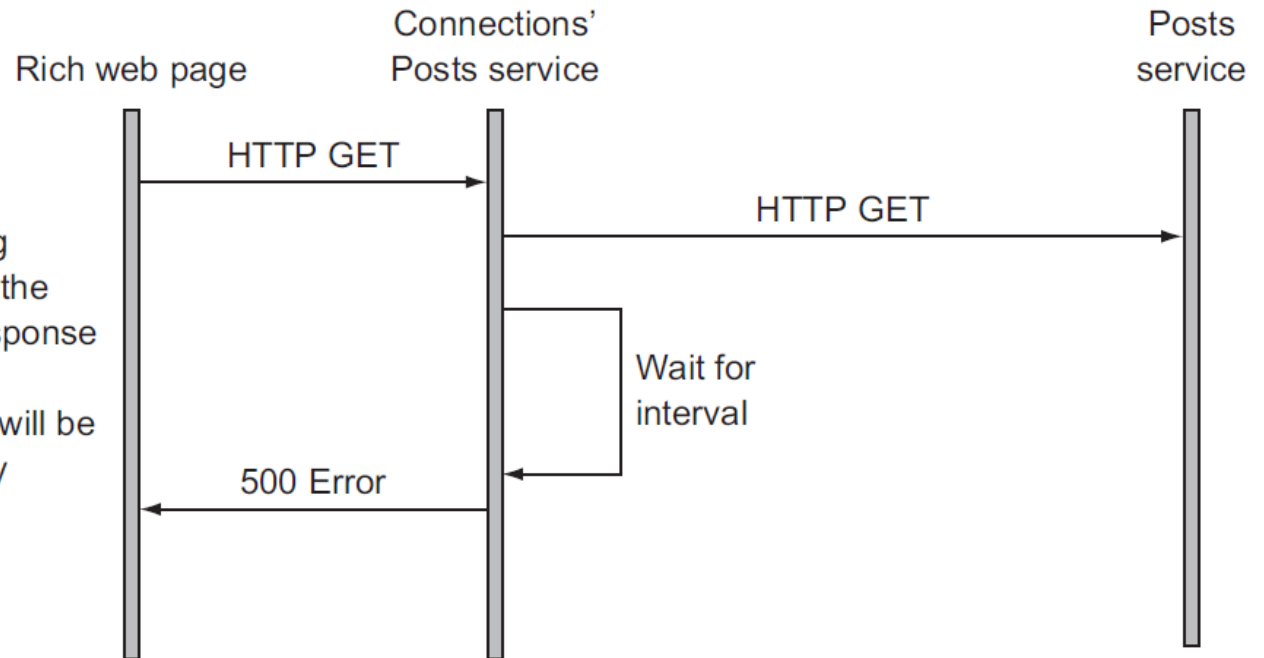
Circuit Breaker

- O circuit breaker ocasionalmente permite chamadas a um serviço degradado. Se essas chamadas tiverem sucesso repetidamente, o circuit breaker será redefinido. Os principais benefícios oferecidos pelo padrão do circuit breaker para chamadas remotas são:
 - **Falhar rapidamente** - Quando um serviço remoto está passando por uma degradação, a aplicação falhará rapidamente e evitará problemas de esgotamento de recursos que geralmente desligam toda a aplicação. *Na maioria das situações de interrupção de serviço, é melhor estar parcialmente indisponível do que completamente indisponível.*
 - **Falhar de forma elegante** - Ao definir um tempo limite e falhar rapidamente, o padrão do circuit breaker nos permite falhar de forma elegante ou procurar mecanismos alternativos para realizar a intenção do usuário. Por exemplo, se um usuário estiver tentando obter dados de uma fonte de dados específica que esteja passando por uma degradação de serviço, nossos serviços podem recuperar esses dados de outra fonte.
 - **Recuperar de forma transparente** - Com o padrão do circuit breaker atuando como intermediário, o circuit breaker pode verificar periodicamente se o recurso solicitado está online novamente e reabilitar o acesso a ele sem intervenção humana.

Circuit Breaker

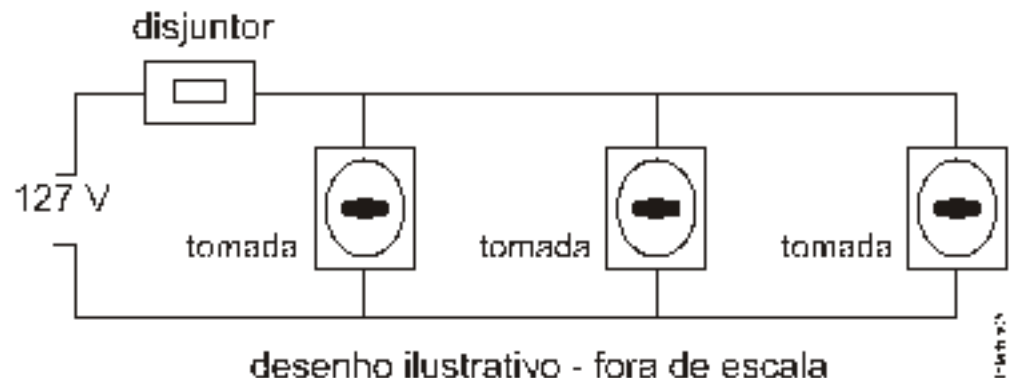
Without circuit breaker

If the Posts service is experiencing trouble, one outcome may be that the client wastes time waiting for a response that will not come. If this happens frequently, the system as a whole will be in a compromised state, with many components simply waiting for something to happen.



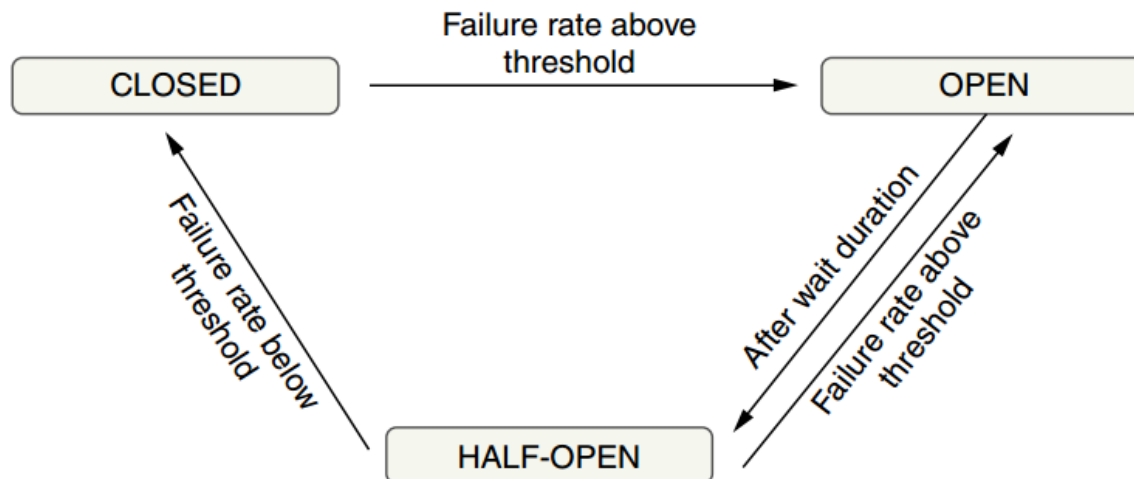
Entendendo os Circuit Breakers

- Para entender os circuit breakers, podemos pensar em sistemas elétricos. O que acontece quando há uma corrente muito alta passando por um fio em um sistema elétrico? Como você deve se lembrar, se o disjuntor detectar essa elevação de corrente, ele interrompe a conexão com o restante do sistema, evitando danos adicionais a outros componentes. O mesmo acontece em nossa arquitetura de código.



Entendendo os Circuit Breakers

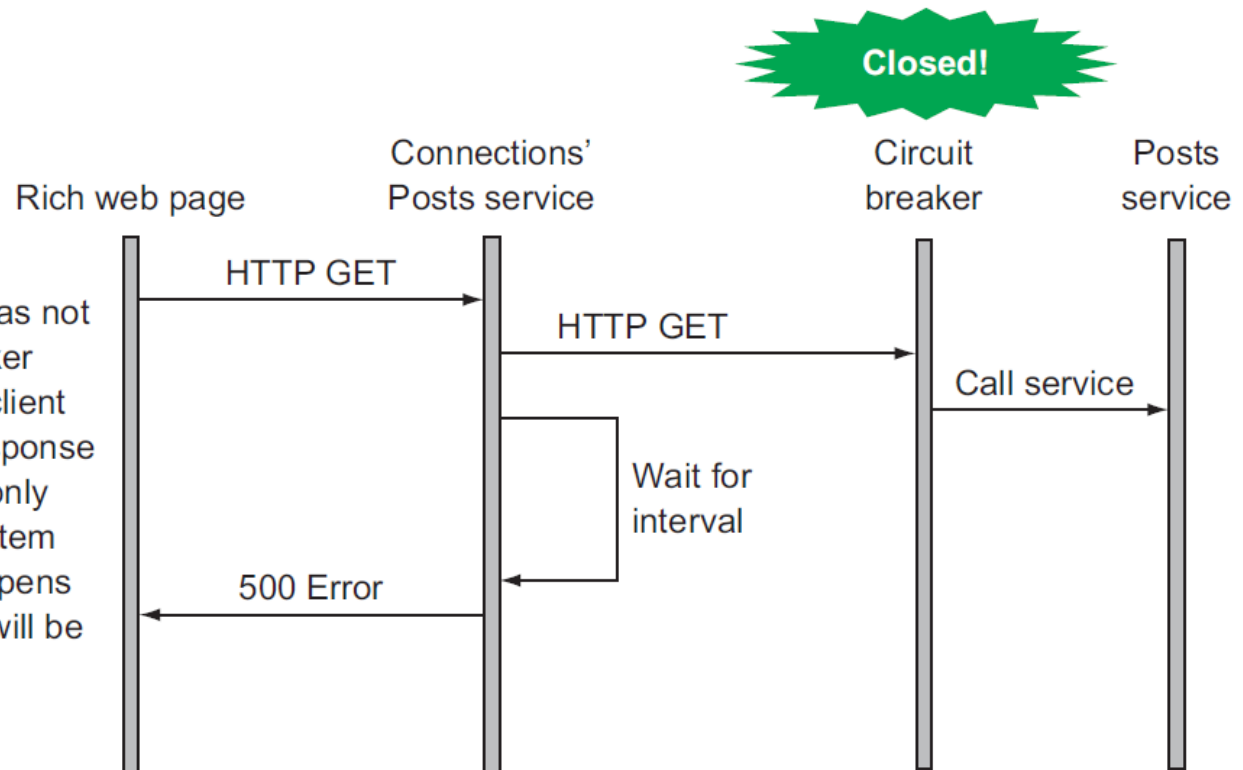
- No Resilience4j, o [circuit breaker](#) é implementado por meio de uma máquina de estados finita com três estados normais.
- Inicialmente, o circuit breaker do Resilience4j começa em um estado fechado e aguarda as solicitações do cliente.



Entendendo os Circuit Breakers

With circuit breaker, in Closed state

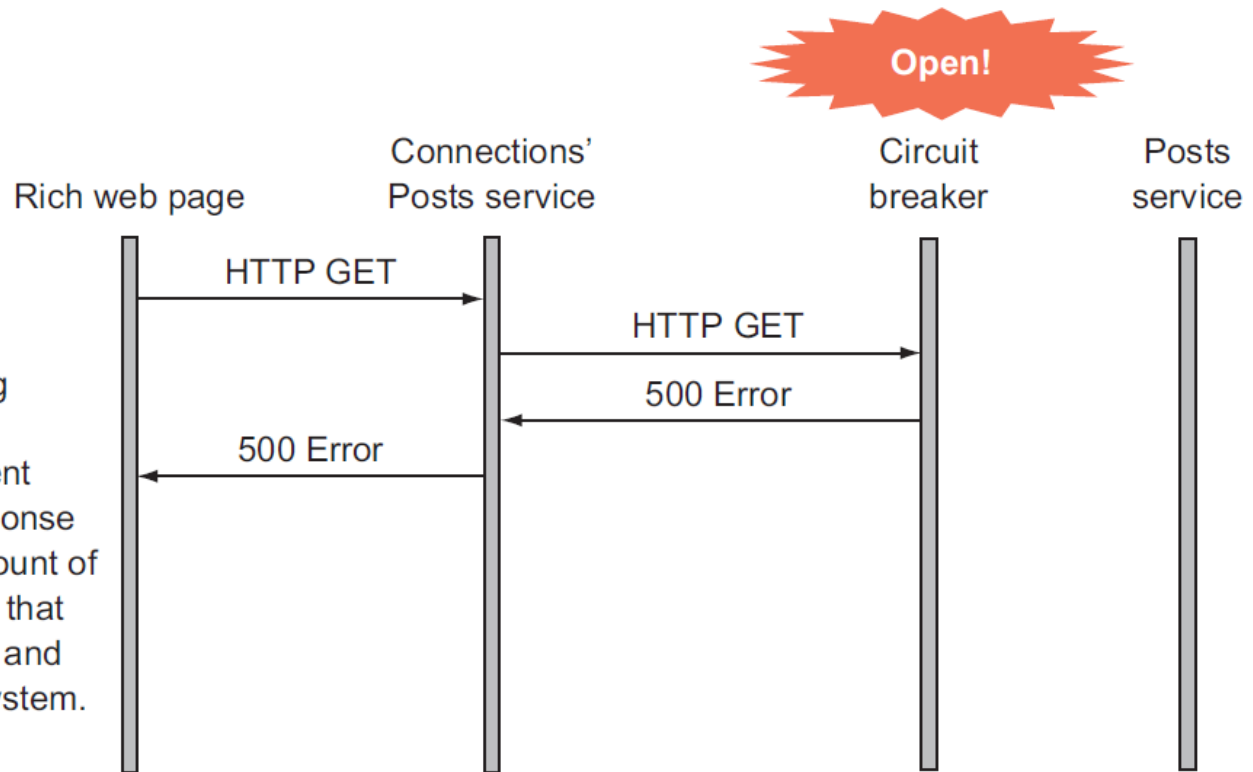
If the Posts service is experiencing trouble—but because the trouble has not yet been detected, the circuit breaker sitting before it is still *closed*—the client may be waiting some time for a response that will not come. If this happens only occasionally, the impact on the system as a whole will be minimal. If it happens frequently, the system as a whole will be in a compromised state, with many components simply waiting for something to happen.



Entendendo os Circuit Breakers

With circuit breaker, in Open state

If the Posts service is experiencing trouble, but has an *open* circuit breaker sitting in front of it, the client will immediately know that no response will be coming. Minimizing the amount of time wasted waiting for responses that will not come increases the health and reliability of the overall software system.

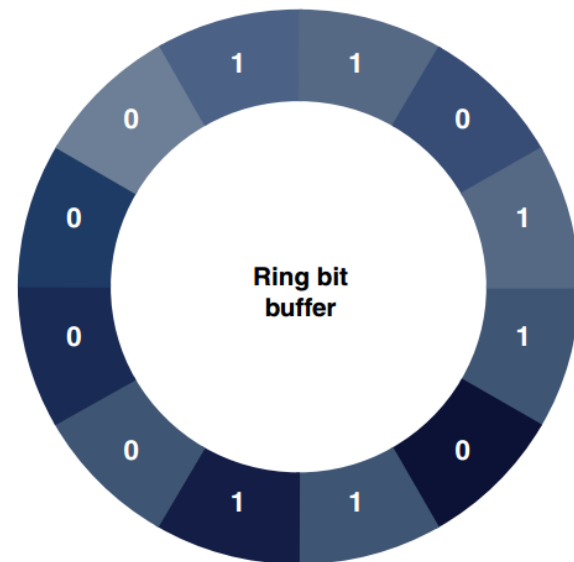


Entendendo os Circuit Breakers

- O **CircuitBreaker** utiliza uma janela deslizante para armazenar e agregar os resultados das chamadas. É possível escolher entre uma janela deslizante baseada em **contagem** ou baseada em **tempo**.
- Prop.: **slidingWindowType** = COUNT_BASED ou TIME_BASED
- A janela deslizante baseada em **contagem** agrega os resultados das últimas **N chamadas**.
- A janela deslizante baseada em **tempo** agrega os resultados das chamadas dos últimos **N segundos**.
- O tamanho da janela é definido pela propriedade **slidingWindowSize**[Exe.: 12 ou 100s]

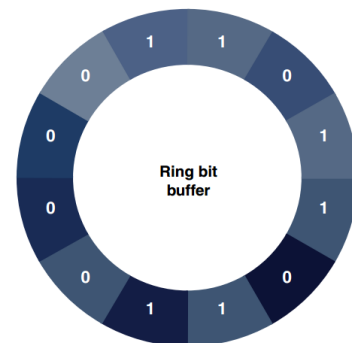
Entendendo os Circuit Breakers

- O estado fechado usa um buffer de bits em anel para armazenar o status de sucesso ou falha das solicitações.
- Quando uma solicitação bem-sucedida é feita, o circuit breaker armazena um bit 0 no buffer de bits em anel.
- Mas se ele não receber uma resposta do serviço invocado, ele armazena um bit 1.
- Prop.: **recordExceptions** é lista de exceções tratadas para registrar o bit 1.



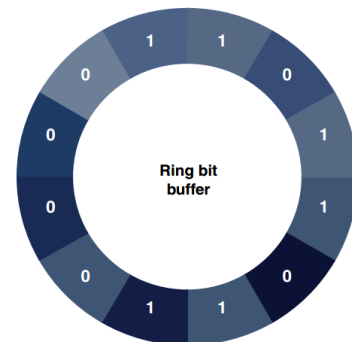
Entendendo os Circuit Breakers

- Para calcular a **taxa de falha**(prop.: **failureRateThreshold** em %), o buffer de bits em anel deve estar cheio.
- Por exemplo, pelo menos 12 chamadas devem ser avaliadas antes que a taxa de falha possa ser calculada.
- Se apenas 11 solicitações forem avaliadas, o circuit breaker não mudará para um estado aberto, mesmo que todas as 11 chamadas falhem.
- Observe que o circuit breaker só abre quando a **taxa de falha** é igual ou superior ao limite configurável. Exe.: se `failureRateThreshold = 50`, o que acontece neste acaso?



Entendendo os Circuit Breakers

- Uma outra situação onde o circuit breakers pode ir para o estado de Open é via a definição das propriedades:
 - `slowCallDurationThreshold`: define em ms o tempo de duração de uma invocação para ser considerada lenta.
 - `slowCallRateThreshold`: define uma percentagem para ser usada como threshold. Quando o número de camadas lentas (definidas pelo `slowCallDurationThreshold`) for igual ou superior a este percentual, o circuit breaker vai para o estado aberto.



Entendendo os Circuit Breakers

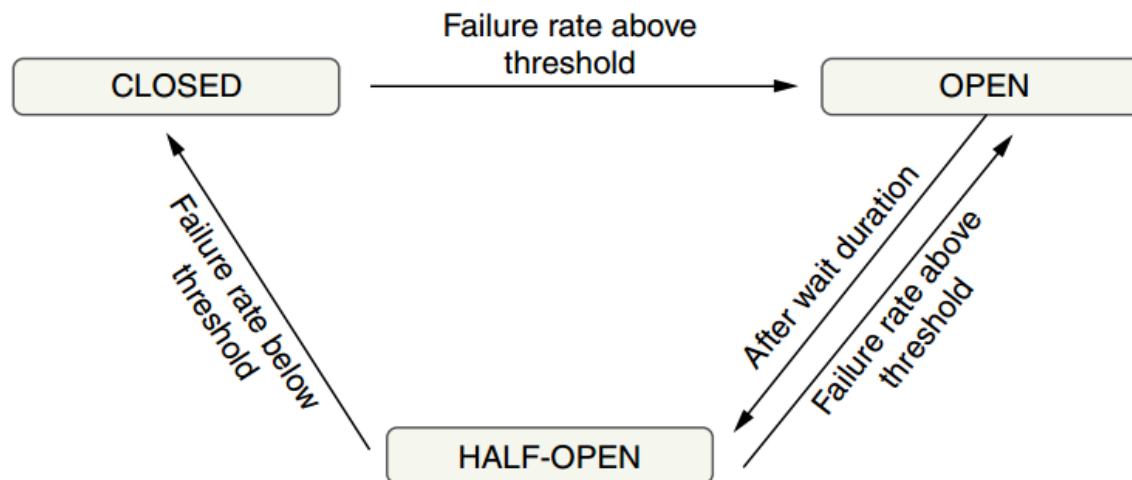
- Quando o circuit breaker está no **estado aberto**, todas as chamadas **são rejeitadas** durante um tempo configurável(prop.: **waitDurationInOpenState**) e o circuit breaker lança uma exceção **CallNotPermittedException**.
- Uma vez que o tempo de **configuração expira**, o circuit breaker muda para o **estado meio-aberto**.

Entendendo os Circuit Breakers

- No estado meio-aberto, o circuit breaker permite um número de solicitações(prop.: **permittedNumberOfCallsInHalfOpenState**) para verificar se o serviço ainda está indisponível. A propriedade **maxWaitDurationInHalfOpenState** pode ser usada para controlar o tempo máximo de permanência no estado **meio-aberto**.
- Se essa nova taxa de falha estiver acima do limite configurado, o circuit breaker volta para o estado aberto; se estiver abaixo ou igual ao limite, ele volta para o estado fechado.

Entendendo os Circuit Breakers

- Resumindo:
 - No estado aberto, o circuit breaker rejeita todas as solicitações.
 - No estado fechado, o circuit breaker aceita todas as solicitações.



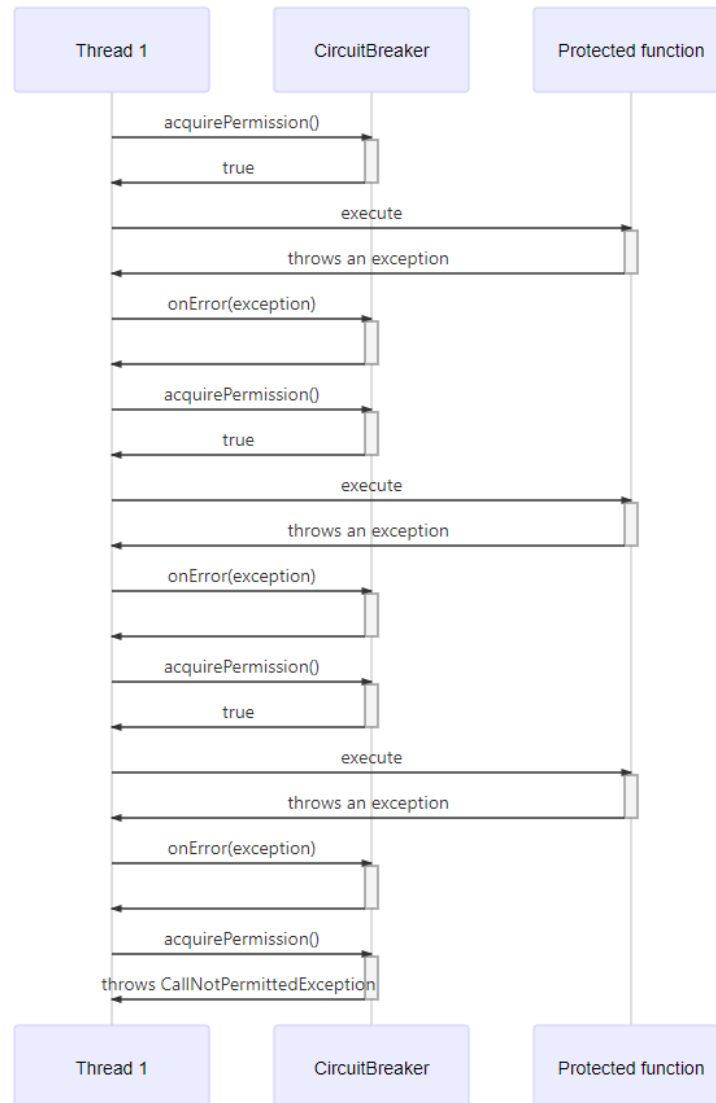
Implementação Interna do CircuitBreaker

- O **CircuitBreaker** é thread safe dado que:
 - O estado do **CircuitBreaker** é armazenado em um **AtomicReference**.
 - O **CircuitBreaker** utiliza operações atômicas para atualizar o estado com funções livres de efeitos colaterais.
 - O registro de chamadas e a leitura de snapshots da janela deslizante são sincronizados. Isso garante a atomicidade, permitindo que apenas uma thread atualize o estado ou a janela deslizante por vez.

Implementação Interna do CircuitBreaker

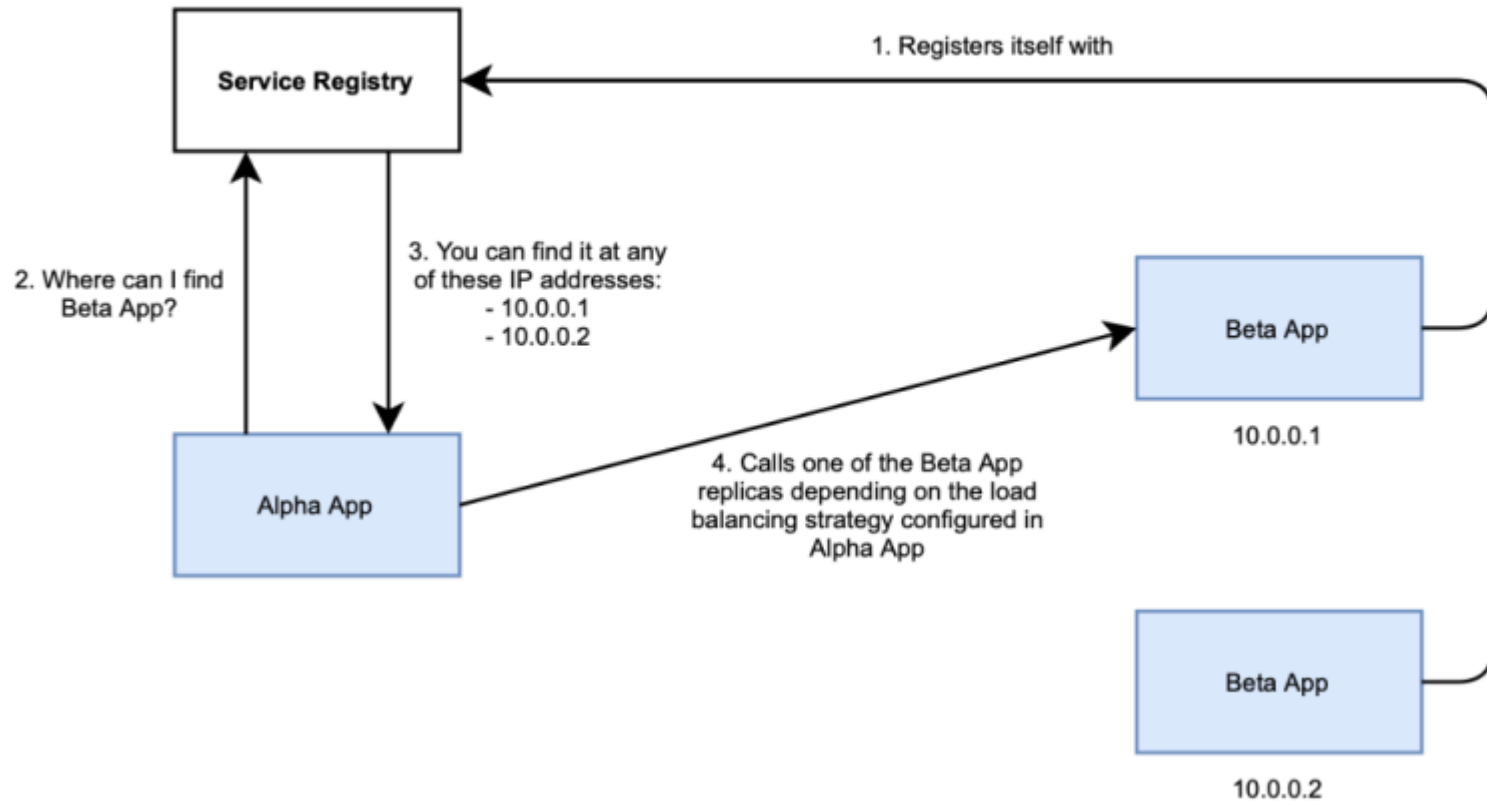
- No entanto, o **CircuitBreaker não sincroniza a execução da função**. Isso significa que a chamada de função em si não faz parte da seção crítica. Caso contrário, o **CircuitBreaker** causaria uma grande penalidade de desempenho e gargalos. Uma chamada de função lenta teria um impacto muito negativo no desempenho/throughput geral.
- Se 20 threads concorrentes solicitarem permissão para executar uma função e o estado do **CircuitBreaker** estiver fechado (**CLOSED**), todas as threads terão permissão para invocar a função, mesmo que o tamanho da janela deslizante seja 15. A janela deslizante **não limita** o número de chamadas simultâneas permitidas.
- Se for necessário restringir o número de threads concorrentes, use um **Bulkhead**. É possível combinar um **Bulkhead** com um **CircuitBreaker**.

Implementação Interna do CircuitBreaker



Cenário de uso do Circuit Breaker

Client-side service discovery and load balancing - Model



Usando Circuit Breakers

```
@RestController
@RequestMapping("/alpha")
public class AlphaController {
    @Autowired
    ServiceAlpha servicealpha;

    @GetMapping
    public ResponseEntity getData() {
        return servicealpha.getData();
    }
}
```

```
@Service
public class ServiceAlpha {

    @Autowired
    RestTemplate restTemplate;

    @CircuitBreaker(name = "servicebeta")
    public ResponseEntity getData() {
        return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
    }
}
```


Usando Circuit Breakers

```
13 management.health.circuitbreakers.enabled: true
14 management:
15   endpoints:
16     web:
17       exposure:
18         include: "*"
19
20 management.endpoint.health:
21   enabled: true
22   show-details: always
23
24
25 resilience4j.circuitbreaker:
26   configs:
27     default:
28       registerHealthIndicator: true
29   instances:
30     servicebeta:
31       slidingWindowType: COUNT_BASED
32       slidingWindowSize: 2
33       waitDurationInOpenState: 10s
34       failureRateThreshold: 50
35       event-consumer-buffer-size: 10
36       registerHealthInticator: true
37
```

Monitorando Circuit Breakers

← → ↻ ⓘ localhost:8082/alpha

OK from 8090

← → ↻ ⓘ localhost:8082/actuator/health/circuitBreakers ☆

Aplicar estilos de formatação ☐

```
{"status":"UP","details":{"servicebeta":{"status":"UP","details":{"failureRate":"0.0%","failureRateThreshold":"50.0%","slowCallRate":"0.0%","slowCallRateThreshold":"100.0%","bufferedCalls":2,"slowCalls":0,"slowFailedCalls":0,"slowlyFailing":false,"state":"CLOSED"}}}}
```

← → ↻ ⓘ localhost:8082/actuator/circuitbreakerevents ☆ ⓘ 📄

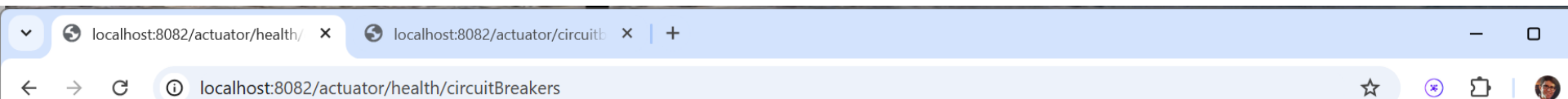
Aplicar estilos de formatação ☐

```
"circuitBreakerEvents":[{"circuitBreakerName":"servicebeta","type":"SUCCESS","creationTime":"2025-01-19T22:13:40.773839900-03:00[America/Fortaleza"],"errorMessage":null,"durationInMs":47,"stateTransition":null},{"circuitBreakerName":"servicebeta","type":"SUCCESS","creationTime":"2025-01-19T22:13:42.182242400-03:00[America/Fortaleza"],"errorMessage":null,"durationInMs":4,"stateTransition":null}]}
```

Monitorando Circuit Breakers

← → ↻ ⓘ localhost:8082/alpha

Fallback in action for circuit breakerNo instances available for SERVICEBETA



Aplicar estilos de formatação

```
{
  "status": "UNKNOWN",
  "details": {
    "servicebeta": {
      "status": "CIRCUIT_OPEN",
      "details": {
        "failureRate": "50.0%",
        "failureRateThreshold": "50.0%",
        "slowCallRate": "0.0%",
        "slowCallRateThreshold": "100.0%",
        "bufferedCalls": 2,
        "slowCalls": 0,
        "slowFailedCalls": 0,
        "failedCalls": 0,
        "notPermittedCalls": 0,
        "state": "OPEN"
      }
    }
  }
}
```



Aplicar estilos de formatação

```
{
  "circuitBreakerEvents": [
    {
      "circuitBreakerName": "servicebeta",
      "type": "SUCCESS",
      "creationTime": "2025-01-19T22:13:40.773839900-03:00[America/Fortaleza]",
      "errorMessage": null,
      "durationInMs": 47,
      "stateTransition": null
    },
    {
      "circuitBreakerName": "servicebeta",
      "type": "SUCCESS",
      "creationTime": "2025-01-19T22:13:42.182242400-03:00[America/Fortaleza]",
      "errorMessage": null,
      "durationInMs": 4,
      "stateTransition": null
    },
    {
      "circuitBreakerName": "servicebeta",
      "type": "ERROR",
      "creationTime": "2025-01-19T22:16:34.322410600-03:00[America/Fortaleza]",
      "errorMessage": "java.lang.IllegalStateException: No instances available for SERVICEBETA",
      "durationInMs": 1,
      "stateTransition": null
    },
    {
      "circuitBreakerName": "servicebeta",
      "type": "FAILURE_RATE_EXCEEDED",
      "creationTime": "2025-01-19T22:16:34.322410600-03:00[America/Fortaleza]",
      "errorMessage": null,
      "durationInMs": null,
      "stateTransition": null
    },
    {
      "circuitBreakerName": "servicebeta",
      "type": "STATE_TRANSITION",
      "creationTime": "2025-01-19T22:16:34.323410700-03:00[America/Fortaleza]",
      "errorMessage": null,
      "durationInMs": null,
      "stateTransition": "CLOSED_TO_OPEN"
    }
  ]
}
```

Usando Circuit Breakers no Spring Gateway

```
spring:
  application:
    name: ServiceGatewayAPI
  cloud:
    loadbalancer:
      ribbon:
        enabled: false
    gateway:
      routes:
        - id: alpha-route
          uri: lb://SERVICEALPHA/alpha
          predicates:
            - Path=/alpha/
          filters:
            - name: Retry
              args:
                retries: 1000
                statuses: BAD_GATEWAY
                methods: GET,POST
                backoff:
                  firstBackoff: 10ms
                  maxBackoff: 50ms
                  factor: 2
                  basedOnPreviousValue: false
            - name: CircuitBreaker
              args:
                name: alphacircuitbreaker
                slidingWindowSize: 20
                permittedNumberOfCallsInHalfOpenState: 5
                failureRateThreshold: 50
                waitDurationInOpenState: 15000
        - id: beta-route
          uri: lb://SERVICEBETA/beta
          predicates:
            - Path=/beta/
```

Importar Dashboard para Monitorar Resilience4J

resilience4j.readme.io/docs/grafana-1

Guides GitHub

v2.2.0 Guides Grafana

ADD-ON MODULES

- Kotlin
- Feign
- RXJAVA 2&3
- Getting Started
- Examples
- SPRING REACTOR
- Getting Started
- Examples
- SPRING BOOT 2&3
- Getting Started
- MICRONAUT
- Getting Started
- SPRING CLOUD
- Getting Started

METRICS

- Micrometer
- Grafana**

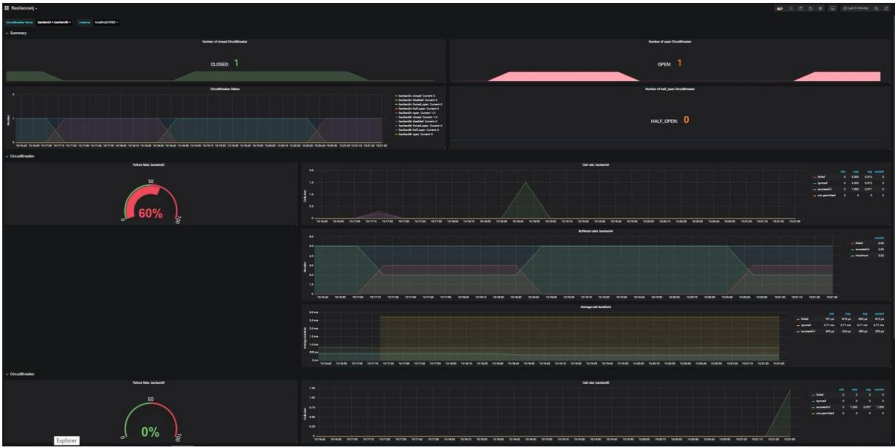
LICENSE

Apache 2.0

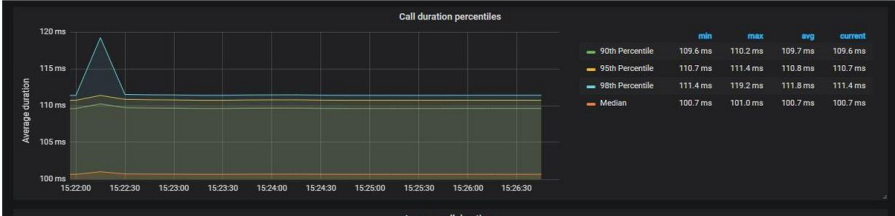
Grafana

Resilience4J provides a [Grafana dashboard](#) for a quick start.

CircuitBreaker Dashboard



Call duration percentiles



	min	max	avg	current
90th Percentile	109.6 ms	110.2 ms	109.7 ms	109.6 ms
95th Percentile	110.7 ms	111.4 ms	110.8 ms	110.7 ms
98th Percentile	111.4 ms	119.2 ms	111.8 ms	111.4 ms
Median	100.7 ms	101.0 ms	100.7 ms	100.7 ms

Importar Dashboard para Monitorar Resilience4J

- Copiar esse código e colocar na importação do grafana

```
resilience4j / grafana_dashboard.json
```

JoranVanBelle issue #1910: Fixed grafana dashboard (#2318) ✓

Code Blame 1262 lines (1262 loc) • 31.2 KB

```
1 {
2   "__inputs": [
3     {
4       "name": "DS_PROMETHEUS",
5       "label": "prometheus",
6       "description": "",
7       "type": "datasource",
8       "pluginId": "prometheus",
9       "pluginName": "Prometheus"
10    }
11  ],
12  "__elements": {},
13  "__requires": [
14    {
15      "type": "grafana",
16      "id": "grafana",
17      "name": "Grafana",
18      "version": "12.0.0"
19    },
20    {
21      "type": "datasource",
22      "id": "prometheus",
23      "name": "Prometheus",
24      "version": "1.0.0"
25    },
26    {
27      "type": "panel",
28      "id": "stat",
29      "name": "Stat",
30      "pluginName": ""
31    }
32  ]
33 }
```

Home > Dashboards > Import dashboard

Import dashboard

Import dashboard from file or Grafana.com


Upload dashboard JSON file
Drag and drop here or click to browse
Accepted file types: .json, .txt

Find and import dashboards for common applications at grafana.com/dashboards

Grafana.com dashboard URL or ID Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_0HnEoN4z",
  "panels": [...]
  ...
}
```

Load Cancel

Monitorando CircuitBreaker

Application ▼ Instance All × ▼ CircuitBreaker Name servicebeta × ▼

Retry Name All × ▼ Bulkhead Name All × ▼

🕒 Last 5 minutes ▼ 🔍 🔄 Refresh ▼

▼ Summary

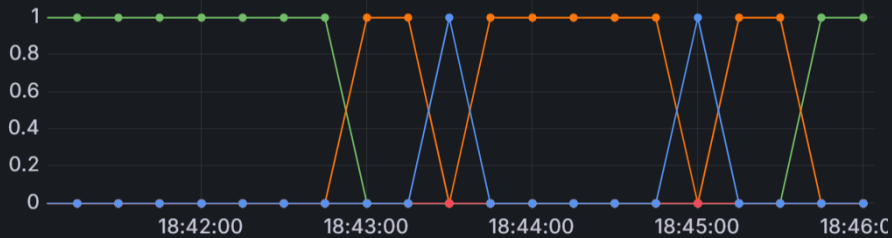
Number of closed CircuitBreaker

1

Number of open CircuitBreaker

0

CircuitBreaker States



Number of half_open CircuitBreaker

0

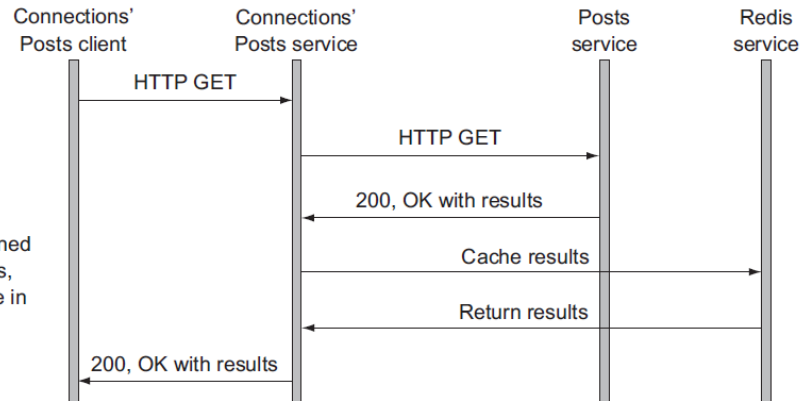
Processamento de Fallback

- Quando uma chamada de serviço remoto falha, em vez de gerar uma exceção, o consumidor do serviço executa um caminho de código alternativo e tenta realizar a ação por outros meios.
- **Os fallbacks fornecem um curso de ação quando um recurso expira o tempo limite ou falha.**

Processamento de Fallback

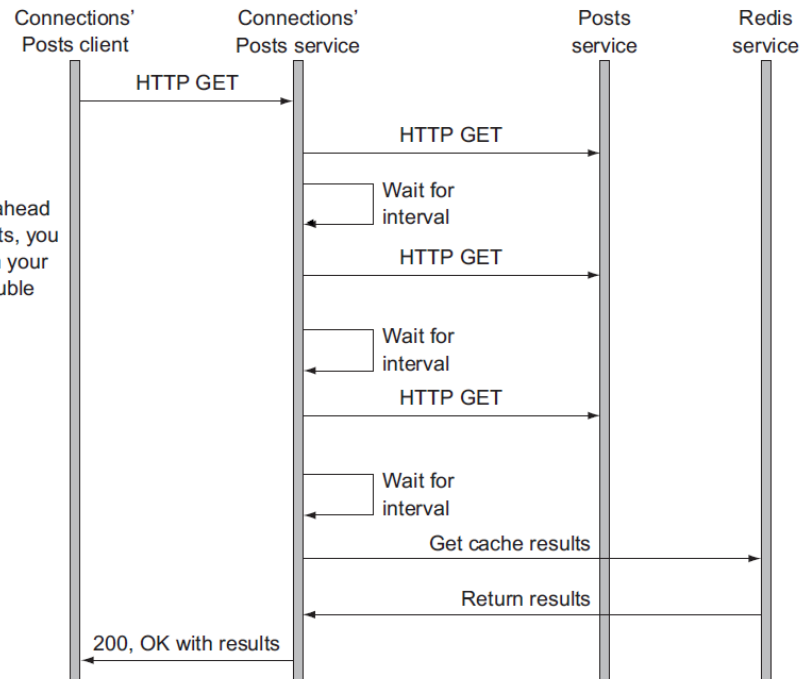
Set up for fallback

When results are returned from the Posts services, save them in the cache in anticipation of future problems.



Implement fallback

Because you thought ahead and cached valid results, you can now utilize them in your fallback logic when trouble occurs.



Processamento de Fallback

- Se você se encontrar usando fallbacks apenas para capturar uma exceção de tempo limite e, em seguida, fazer apenas o registro do erro, você deve usar um bloco try...catch padrão em torno da invocação do serviço: capture a exceção e coloque a lógica de registro dentro do bloco try...catch.
- Esteja ciente das ações que você realiza com suas funções de fallback.
- Se você chamar outro serviço distribuído em seu serviço de fallback, talvez seja necessário envolver o fallback com um `@CircuitBreaker`.
- Lembre-se de que a mesma falha que você está experimentando em sua ação principal também pode afetar sua opção de fallback secundária. Crie seu código de forma defensiva!

Processamento de Fallback no Service

```
@CircuitBreaker(name = "servicebeta",
    fallbackMethod= "buildFallbackGetData")
public ResponseEntity getData() {
    return restTemplate.getForEntity("http://SERVICEBETA/beta", String.class);
}
public ResponseEntity buildFallbackGetData(Throwable t) {
    return ResponseEntity.ok("Fallback in action");
}
```

Processamento de Fallback na Interface OpenFeign

```
@FeignClient("servicebeta")
public interface BetaServiceInterface {

    @RequestMapping(method= RequestMethod.GET, value="/beta", consumes="application/json")
    @CircuitBreaker(name = "servicebeta", fallbackMethod= "getDataFallback")
    String getData();

    default String getDataFallback(Throwable exception) {
        return "00000-00000";
    }
}
```